

Desarrollo Web

Desarrollo Web con Flask,

Requests y Swagger

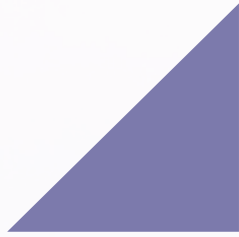




introducción

El desarrollo web existen herramientas que ayudan a crear aplicaciones mucho más sencillo y rápido. Una de ellas es Flask, un framework de Python que destaca por ligero y flexible, permitiendo aplicaciones web con muy pocas líneas de código. Junto a Flask, es fundamental entender cómo funcionan los requests, que son las peticiones que permiten la comunicación entre el cliente y el servidor mediante métodos como GET, POST, PUT y DELETE. Estos métodos son la base de cualquier interacción en la web, desde consultar información hasta crear, actualizar o eliminar datos.

Por otro lado, cuando hablamos de APIs, surge la necesidad de documentarlas y probarlas de manera clara y accesible, y ahí entra Swagger, una herramienta que facilita la creación de documentación interactiva y que se integra perfectamente con Flask. Finalmente, veremos cómo conectar Flask con MySQL, lo que nos permitirá construir aplicaciones completas que no solo procesen información, sino que también la almacenen en una base de datos robusta.



Flask: Microframework

Definición: Microframework Python ligero y flexible que permite crear aplicaciones web de forma rápida y sencilla.

Características clave:

- Fácil de aprender y configurar
- Altamente extensible con plugins
- Soporte para rutas y templates Jinja2
- Servidor de desarrollo integrado

Casos de uso comunes:

- Desarrollo de APIs RESTful
- Aplicaciones web simples y prototipos
- Microservicios y backends ligeros

Template en Flask

Archivo HTML que se usa para mostrar contenido dinámico en una aplicación web. En lugar de escribir páginas estáticas, Flask utiliza el motor de plantillas Jinja2, que permite insertar variables, condicionales y bucles dentro del HTML.

- Un template es la parte visual de la aplicación (lo que ve el usuario).
- Se conecta con la lógica de Flask para mostrar datos que vienen del servidor.
- Permite separar la lógica del diseño, haciendo el código más ordenado y fácil de mantener.

Iniciar aplicación

- Flask

Archivo HTML que se usa para mostrar contenido dinámico en una aplicación web. En lugar de escribir páginas estáticas, Flask utiliza el motor de plantillas Jinja2, que permite insertar variables, condicionales y bucles dentro del HTML.

- Un template es la parte visual de la aplicación (lo que ve el usuario).
- Se conecta con la lógica de Flask para mostrar datos que vienen del servidor.
- Permite separar la lógica del diseño, haciendo el código más ordenado y fácil de mantener.

Requests

- Es una solicitud que viaja desde el cliente (por ejemplo, tu navegador o una aplicación) hacia el servidor.
- El servidor recibe esa solicitud, la procesa y devuelve una respuesta (response).
- Los requests pueden incluir parámetros, encabezados, datos de autenticación o instrucciones sobre qué operación realizar.

requests

Requests:

Tipos-usos

- GET → pedir datos sin modificarlos (ej. cargar una página).
- POST → enviar datos para crear o procesar algo nuevo (ej. enviar un formulario).
- PUT → reemplazar un recurso completo (ej. actualizar todo un perfil).
- PATCH → modificar solo una parte de un recurso (ej. cambiar un correo en el perfil).
- DELETE → eliminar un recurso (ej. borrar un comentario).
- HEAD → pedir solo los encabezados, sin contenido (ej. verificar si un archivo existe).
- OPTIONS → consultar qué métodos acepta el servidor (ej. saber si permite POST).
- CONNECT → crear un túnel de comunicación, usado en proxies.
- TRACE → mostrar la ruta que sigue la petición, útil para diagnóstico.

requests

Requests:

Tipos-usos

- GET → pedir datos sin modificarlos (ej. cargar una página).
- POST → enviar datos para crear o procesar algo nuevo (ej. enviar un formulario).
- PUT → reemplazar un recurso completo (ej. actualizar todo un perfil).
- PATCH → modificar solo una parte de un recurso (ej. cambiar un correo en el perfil).
- DELETE → eliminar un recurso (ej. borrar un comentario).
- HEAD → pedir solo los encabezados, sin contenido (ej. verificar si un archivo existe).
- OPTIONS → consultar qué métodos acepta el servidor (ej. saber si permite POST).
- CONNECT → crear un túnel de comunicación, usado en proxies.
- TRACE → mostrar la ruta que sigue la petición, útil para diagnóstico.

requests

Funcionamiento de:

◆ GET

- Función: Pedir datos al servidor sin modificarlos.
- Cómo trabaja: El cliente envía la URL y parámetros opcionales; el servidor responde con el recurso solicitado (HTML, JSON, imagen, etc.).
- Ejemplo: Cargar una página web o consultar una lista de productos.

◆ POST

- Función: Enviar datos al servidor para crear o procesar algo nuevo.
- Cómo trabaja: El cliente envía información en el cuerpo del request (formulario, archivo, JSON). El servidor la procesa y devuelve un resultado.
- Ejemplo: Registrar un usuario, enviar un formulario de contacto o subir una foto.

◆ PUT

- Función: Reemplazar un recurso existente con nuevos datos.
- Cómo trabaja: El cliente envía un objeto completo que sustituye al recurso previo. El servidor guarda la nueva versión.
- Ejemplo: Actualizar todo el perfil de un usuario en una aplicación.



Swagger

Swagger es un conjunto de herramientas de código abierto que sirve para diseñar, documentar y probar APIs de manera clara, interactiva y estandarizada. Se basa en la especificación OpenAPI, lo que permite describir cómo funciona una API de forma legible tanto para humanos como para máquinas.

swagger

Rest API con swagger

◆ Funcionamiento

1. Diseño de la API

- Defines los endpoints (ej. /usuarios, /productos) y los métodos HTTP que aceptan (GET, POST, PUT, DELETE).
- Especificas parámetros, cuerpos de petición y respuestas esperadas.

2. Documentación automática

- Swagger genera una interfaz visual (Swagger UI) donde ves todos los endpoints descritos.
- Cada endpoint muestra qué datos espera y qué devuelve.

3. Pruebas interactivas

- Desde Swagger UI puedes ejecutar los requests directamente (ej. probar un POST /usuarios enviando un JSON).
- Esto evita tener que usar herramientas externas como Postman.

4. Generación de código

- Con Swagger Codegen puedes crear clientes o servidores en distintos lenguajes basados en la especificación

Swagger

1. Instalar dependencias

En tu entorno de Python instala Flask y Flasgger (o flask-swagger-ui):

2. Crear una aplicación Flask básica

3. Documentar endpoints con docstrings

- Dentro de cada función puedes añadir un bloque de documentación en formato YAML.
- Swagger lo interpreta y genera automáticamente la documentación interactiva.

4. Acceder a Swagger UI

- Una vez corras tu aplicación (python app.py), abre en el navegador: <http://127.0.0.1:5000/apidocs/> (127.0.0.1 in Bing).
- Allí verás la documentación generada y podrás probar los endpoints con botones interactivos.

